

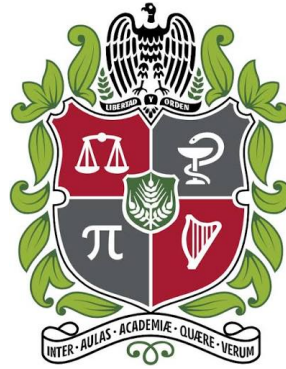
PROYECTO FINAL MATEMÁTICAS DISCRETAS II REPORTE TÉCNICO

Elaborado por:

Jedric Julián Montealegre Contreras
Diego Alejandro Reyes Cárdenas
David Eduardo Calderón Parra

Presentado a:

Arles Ernesto Rodríguez Portela



UNIVERSIDAD NACIONAL DE COLOMBIA
FACULTAD DE INGENIERÍA
MATEMÁTICAS DISCRETAS II
BOGOTÁ D.C.
2026

Resumen

Este proyecto analiza el rendimiento de algoritmos recursivos combinando modelos matemáticos y mediciones experimentales. Se implementaron MergeSort y dos variantes de QuickSort en Java: una con particiones aproximadamente balanceadas y otra diseñada para reproducir el peor caso sobre entradas ordenadas. Las ecuaciones de recurrencia se resolvieron para establecer los órdenes de crecimiento esperados, mientras que la implementación registró tiempo de ejecución, comparaciones, escrituras, profundidad máxima de recursión y variación aproximada del heap. Los resultadosaron un comportamiento cercano a $n \log_2(n)$ para MergeSort y QuickSort balanceado, y cercano a n^2 para QuickSort en peor caso. Además, una prueba controlada con una pila de 1 MB mostró que la variante no balanceada completó $n = 7500$ y produjo `StackOverflowError` en $n = 8000$. El estudio demuestra que la notación asintótica es necesaria para comparar algoritmos, pero debe complementarse con constantes, profundidad de pila y condiciones concretas de ejecución para apoyar decisiones de ingeniería.

Palabras clave: Algoritmos recursivos, ecuaciones de diferencias, recurrencias, MergeSort, QuickSort, complejidad computacional, Java, pila de llamadas, benchmarking.

Índice

1. Introducción y motivación	1
2. Problema abordado	1
3. Objetivos	1
3.1. Objetivo general	1
3.2. Objetivos específicos	2
4. Fundamentos teóricos	2
4.1. Ecuaciones de diferencias y recurrencias	2
4.2. MergeSort	2
4.3. QuickSort balanceado y peor caso	2
4.4. Profundidad de recursión y memoria	2
4.5. Modelo de ajuste	3
5. Diseño de la solución	3
5.1. Componentes	3
5.2. Organización del repositorio	3
6. Algoritmos, estructuras y modelos utilizados	4
6.1. Estructuras de datos	4
6.2. Instrumentación	4
6.3. Escenarios de prueba	4
7. Metodología experimental	4
7.1. Tamaños utilizados	4
7.2. Criterios de análisis	5
8. Resultados obtenidos	5
8.1. Ajuste de los modelos	5
8.2. Gráficas de Rendimiento Temporal y Espacial	6
8.3. Comparación de tiempos y operaciones	6
8.4. Profundidad y límite de pila	6
8.5. Medición de heap	6
9. Discusión	7
10. Limitaciones y posibles mejoras	8
10.1. Limitaciones	8
10.2. Posibles mejoras	8
11. Conclusiones	8
Anexo A. Instrucciones de ejecución	10

1. Introducción y motivación

El análisis de algoritmos suele comenzar con la notación asintótica, especialmente Big-O, porque permite describir cómo crece el costo computacional cuando aumenta el tamaño de la entrada. Este enfoque facilita comparar soluciones sin depender de una máquina específica. Sin embargo, dos implementaciones con el mismo orden asintótico pueden presentar tiempos muy diferentes debido a sus constantes, al patrón de acceso a memoria, a la selección de pivotes, a la compilación JIT y a la profundidad de la pila de llamadas [1, 2].

La motivación del proyecto consiste en reducir la distancia entre el análisis teórico y la ejecución real. En sistemas con restricciones de tiempo o memoria no basta con afirmar que un algoritmo pertenece a $\mathcal{O}(n \log n)$; también interesa estimar el costo esperado para tamaños concretos, conocer la memoria auxiliar requerida y determinar si la recursión puede superar el límite de pila. La presentación inicial del proyecto identifica precisamente estas necesidades: latencia de hardware, consumo de stack y predicción de puntos de quiebre antes de un fallo. Por esta razón, se construyó una implementación reproducible en Java que instrumenta los algoritmos y exporta resultados en formato CSV. El objetivo no es reemplazar la teoría por mediciones aisladas, sino integrar ambos enfoques: primero se obtiene un modelo mediante ecuaciones de diferencias y luego se contrasta con evidencia experimental.

2. Problema abordado

El problema central es determinar en qué medida las recurrencias teóricas describen el comportamiento observable de algoritmos recursivos y cómo influyen la forma de la entrada y la estrategia de partición en el tiempo y la profundidad de ejecución. Para responderlo se plantearon cuatro preguntas técnicas:

- ¿El tiempo medido de MergeSort se ajusta a una función proporcional a $n \log_2(n)$?
- ¿QuickSort conserva ese comportamiento cuando las particiones son aproximadamente balanceadas?
- ¿Qué ocurre cuando QuickSort recibe una entrada que genera particiones de tamaños $n - 1$ y 0 ?
- ¿Cómo se relaciona la profundidad de recursión con el riesgo de `StackOverflowError`?

El alcance se limitó a arreglos de enteros, ejecución monohilo y tres escenarios controlados. No se pretende establecer tiempos universales, pues los valores absolutos dependen del hardware y la JVM; se busca validar tendencias, comparar estructuras recursivas y dejar una base extensible para experimentos posteriores.

3. Objetivos

3.1. Objetivo general

Desarrollar y validar un modelo teórico-experimental que permita predecir y comparar el comportamiento temporal y espacial de algoritmos recursivos de ordenamiento mediante ecuaciones de diferencias e instrumentación en Java.

3.2. Objetivos específicos

- Formular y resolver las recurrencias asociadas con MergeSort y QuickSort.
- Implementar versiones instrumentadas que registren tiempo, operaciones y profundidad recursiva.
- Diseñar pruebas reproducibles para casos aleatorios, balanceados y de peor caso.
- Ajustar constantes de proporcionalidad para las bases $n \log_2(n)$ y n^2 .
- Identificar limitaciones de medición y proponer mejoras metodológicas y de implementación.

4. Fundamentos teóricos

4.1. Ecuaciones de diferencias y recurrencias

Una ecuación de recurrencia (o ecuación de diferencias discretas en el análisis algorítmico) define el costo de un problema de tamaño n en función del costo de subproblemas más pequeños [3, 4]. En algoritmos de divide y vencerás suele emplearse la forma:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n) \quad (1)$$

Donde a es el número de subproblemas, n/b es el tamaño de cada uno y $f(n)$ representa el trabajo de división y combinación. El Teorema Maestro permite clasificar muchas recurrencias de esta forma comparando $f(n)$ con $n^{\log_b(a)}$ [1, 5].

4.2. MergeSort

MergeSort divide el arreglo en dos mitades, ordena cada mitad de forma recursiva y combina los resultados en tiempo lineal. Para n potencia de dos, su recurrencia puede expresarse como:

$$T(n) = 2T\left(\frac{n}{2}\right) + cn, \quad T(1) = d \quad (2)$$

Al expandirla o resolverla por inducción se obtienen $\log_2(n)$ niveles con trabajo lineal por nivel, de modo que $T(n) = cn \log_2(n) + dn$, y por tanto, $T(n) \in \Theta(n \log n)$ [1].

4.3. QuickSort balanceado y peor caso

Cuando el pivote divide el arreglo en dos partes de tamaños semejantes, QuickSort presenta una recurrencia equivalente a $T(n) = 2T(n/2) + cn$ y su costo es $\Theta(n \log n)$ [2]. En cambio, si el pivote resulta ser siempre el mayor o el menor elemento, la recurrencia se aproxima a:

$$T(n) = T(n-1) + cn \quad (3)$$

Su expansión mediante sumas discretas algebraicas produce una sumatoria aritmética proporcional a $\frac{n(n-1)}{2}$, por lo que el costo es cuadrático: $\Theta(n^2)$ [3, 6].

4.4. Profundidad de recursión y memoria

La profundidad de la pila depende de la forma del árbol de recursión. MergeSort y QuickSort balanceado generan una altura aproximada de $\log_2(n) + 1$. QuickSort en peor caso genera una cadena de llamadas de longitud cercana a n . MergeSort utiliza además un arreglo auxiliar de tamaño n , mientras que QuickSort es esencialmente *in-place*, aunque consume

espacio en la pila de llamadas para rastrear los límites de las particiones. En consecuencia, las complejidades espaciales esperadas son $\mathcal{O}(n)$ para MergeSort, $\mathcal{O}(\log n)$ para QuickSort balanceado y $\mathcal{O}(n)$ para QuickSort en peor caso debido al stack de llamadas [1, 2].

4.5. Modelo de ajuste

Para relacionar teoría y medición se utilizó el modelo $t(n) \approx \alpha g(n)$, donde $g(n)$ es $n \log_2(n)$ o n^2 . La constante α se estimó por mínimos cuadrados sin intercepto:

$$\alpha = \frac{\sum g(n_i)t_i}{\sum g(n_i)^2} \quad (4)$$

La calidad del ajuste se resumió mediante el coeficiente de determinación R^2 . Un valor cercano a 1 indica que la base teórica explica una proporción alta de la variación temporal observada.

5. Diseño de la solución

La solución se organizó en componentes independientes para facilitar su prueba y extensión. El flujo parte de la generación de entradas, continúa con la ejecución de algoritmos instrumentados y termina con la exportación de métricas y el ajuste de los modelos. La arquitectura general sigue una secuencia lineal (Generador $\rightarrow$ Algoritmos $\rightarrow$ Recolector $\rightarrow$ Benchmark $\rightarrow$ Exportación y regresión).

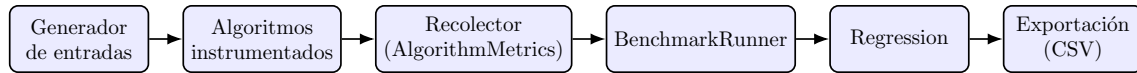


Figura 1: Flujo general de la solución experimental.

5.1. Componentes

Componente	Responsabilidad
Generador de entradas	Construye arreglos pseudoaleatorios u ordenados con semillas deterministas.
Algoritmos instrumentados	Implementa MergeSort, QuickSort con pivote central y QuickSort con último pivote.
AlgorithmMetrics	Registra comparaciones, escrituras, profundidad máxima y variación aproximada del heap.
BenchmarkRunner	Realiza calentamientos, repeticiones, ordenamiento de tiempos y cálculo de la mediana.
Regression	Ajusta la constante α y calcula R^2 para cada escenario.
Exportación	Genera un CSV que puede analizarse con hojas de cálculo o lenguajes de visualización.

Tabla 1: Componentes de la arquitectura de la solución experimental.

5.2. Organización del repositorio

El repositorio incluye código principal, pruebas, scripts para Windows y Linux, datos de referencia, gráficas, reporte técnico y presentación base. No requiere Maven ni Gradle, lo que reduce dependencias y permite compilar directamente con `javac`. La organización sigue una separación convencional entre `src/main/java` y `src/test/java`.

6. Algoritmos, estructuras y modelos utilizados

6.1. Estructuras de datos

La estructura principal es un arreglo de enteros nativo `int[]`. MergeSort crea un arreglo auxiliar del mismo tamaño para combinar subarreglos; QuickSort reorganiza el arreglo original mediante intercambios directos (*swaps*). Las métricas se almacenan en objetos simples y los resultados de cada tamaño se representan mediante registros inmutables (`record`) de Java.

6.2. Instrumentación

La recolección de métricas se realiza de la siguiente manera:

- **Tiempo:** Diferencia entre dos lecturas de `System.nanoTime()`.
- **Profundidad:** Contador incrementado al entrar en una llamada recursiva y decrementado al salir.
- **Operaciones:** Conteo explícito de comparaciones y escrituras sobre el arreglo.
- **Heap:** Diferencia entre el máximo observado de `totalMemory()` - `freeMemory()` y una línea base previa.
- **Validación:** Comprobación de que el arreglo final queda ordenado en cada repetición.

6.3. Escenarios de prueba

Algoritmo	Escenario	Entrada	Base temporal	Profundidad esperada
MergeSort	Aleatorio	Arreglo pseudoaleatorio	$n \log_2(n)$	$\log_2(n) + 1$
QuickSort	Balanceado	Ordenado, pivote central	$n \log_2(n)$	$\log_2(n) + 1$
QuickSort	Peor caso	Ordenado, último pivote	n^2	n

Tabla 2: Diseño de los escenarios de prueba controlados.

7. Metodología experimental

La corrida de referencia se efectuó con OpenJDK 21.0.10 sobre Linux amd64. Para cada tamaño se realizaron tres ejecuciones de calentamiento y siete ejecuciones medidas. El tiempo representativo fue la mediana, por ser menos sensible a valores atípicos que el promedio. Antes de cada repetición se solicitó una recolección de basura mediante `System.gc()` y se introdujo una pausa corta para reducir el ruido externo.

7.1. Tamaños utilizados

MergeSort y QuickSort balanceado se evaluaron entre $n = 128$ y $n = 16384$, duplicando el tamaño en cada paso. QuickSort en peor caso se evaluó hasta $n = 4096$ para evitar que una falla de pila interrumpiera el benchmark principal. Adicionalmente, se ejecutó una prueba independiente de límite de stack con la directiva `-Xss1m`, incrementos de 500 y un máximo de 20000.

7.2. Criterios de análisis

Los criterios de evaluación de los resultados fueron:

1. Crecimiento del tiempo mediano frente al tamaño de entrada.
2. Correspondencia entre la profundidad observada y la predicción teórica.
3. Ajuste de la base teórica mediante α y R^2 .
4. Comparaciones y escrituras como métricas independientes del reloj.
5. Detección experimental del punto de `StackOverflowError`.

8. Resultados obtenidos

8.1. Ajuste de los modelos

Escenario	Base teórica	Coefficiente α	Coefficiente R^2
MergeSort aleatorio	$n \log_2(n)$	12.2777 ns/unidad	0.9990
QuickSort balanceado	$n \log_2(n)$	3.2315 ns/unidad	0.9903
QuickSort peor caso	n^2	0.3238 ns/unidad	0.9969

Tabla 3: Resultados del ajuste de modelos por mínimos cuadrados.

Los tres valores de R^2 fueron superiores a 0.99, lo que indica que las bases teóricas de las ecuaciones de diferencias describen con alta fidelidad la tendencia observada.

8.2. Gráficas de Rendimiento Temporal y Espacial

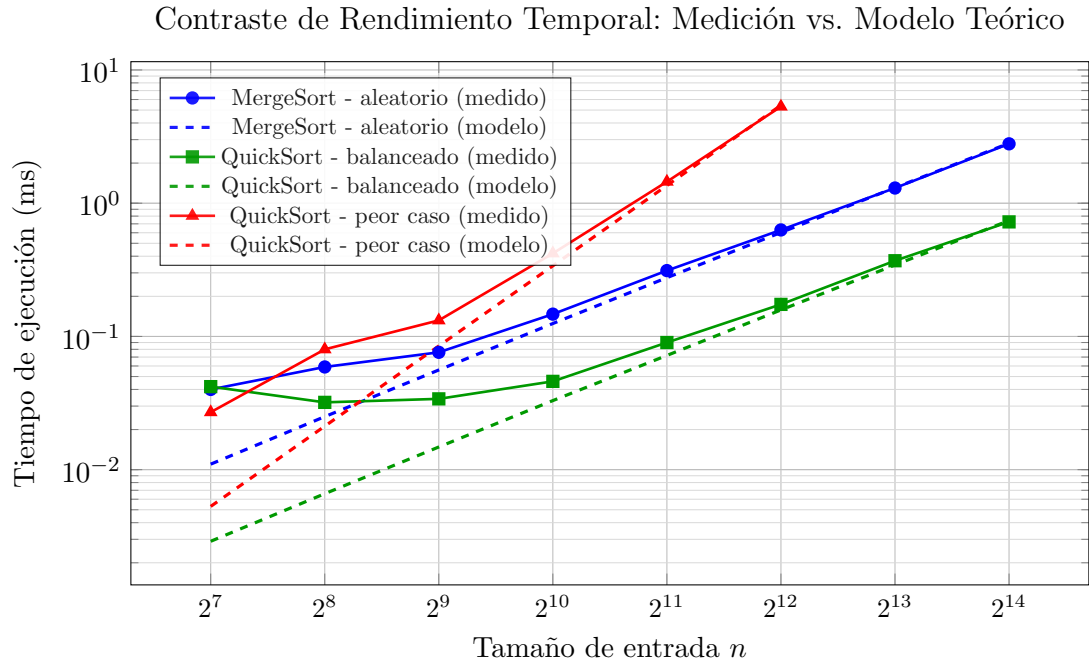


Figura 2: Curvas de escala temporal reales vs. teóricas ajustadas en escala logarítmica bilineal.

8.3. Comparación de tiempos y operaciones

Escenario	n	Tiempo mediano (ms)	Profundidad	Comparaciones	Escrituras
MergeSort aleatorio	16384	2.790	15	208,685	458,752
QuickSort balanceado	16384	0.724	15	196,624	32,768
QuickSort peor caso	4096	5.310	4,096	8,386,560	0

Tabla 4: Comparación detallada de operaciones y recursos consumidos.

Para $n = 16384$, MergeSort registró aproximadamente 2.790 ms y QuickSort balanceado 0.724 ms. Ambos alcanzaron la profundidad de pila esperada de 15, coherente con $\log_2(16384) + 1$. En el peor caso, QuickSort necesitó 5.310 ms para un tamaño de apenas $n = 4096$, efectuando más de 8 millones de comparaciones y alcanzando una profundidad de 4096 llamadas.

8.4. Profundidad y límite de pila

La diferencia espacial fue mucho más marcada que la temporal. En los escenarios balanceados, duplicar el tamaño n incrementó la profundidad exactamente en una unidad. En cambio, en el peor caso de QuickSort cada elemento añadió un marco de pila adicional, produciendo una profundidad lineal $\mathcal{O}(n)$. Con la opción `-Xss1m`, la prueba independiente de pila completó con éxito $n = 7500$ y falló con `StackOverflowError` en $n = 8000$.

8.5. Medición de heap

La medición basada en `Runtime` presentó deltas iguales a cero en la mayoría de las corridas cortas, registrando un incremento neto detectable de 65,552 bytes únicamente

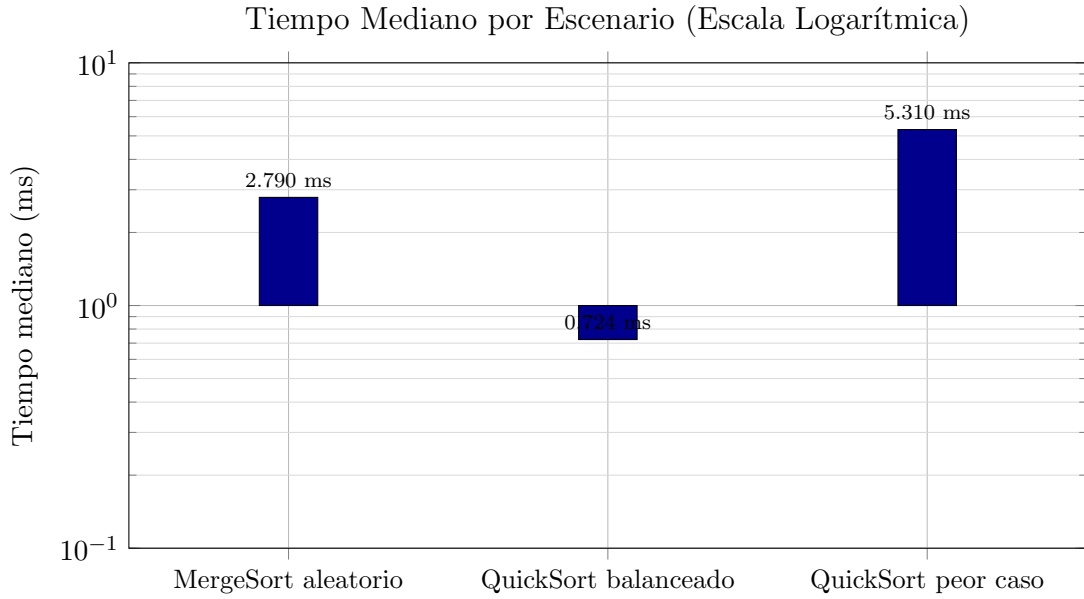


Figura 3: Tiempo mediano por escenario en escala logarítmica.

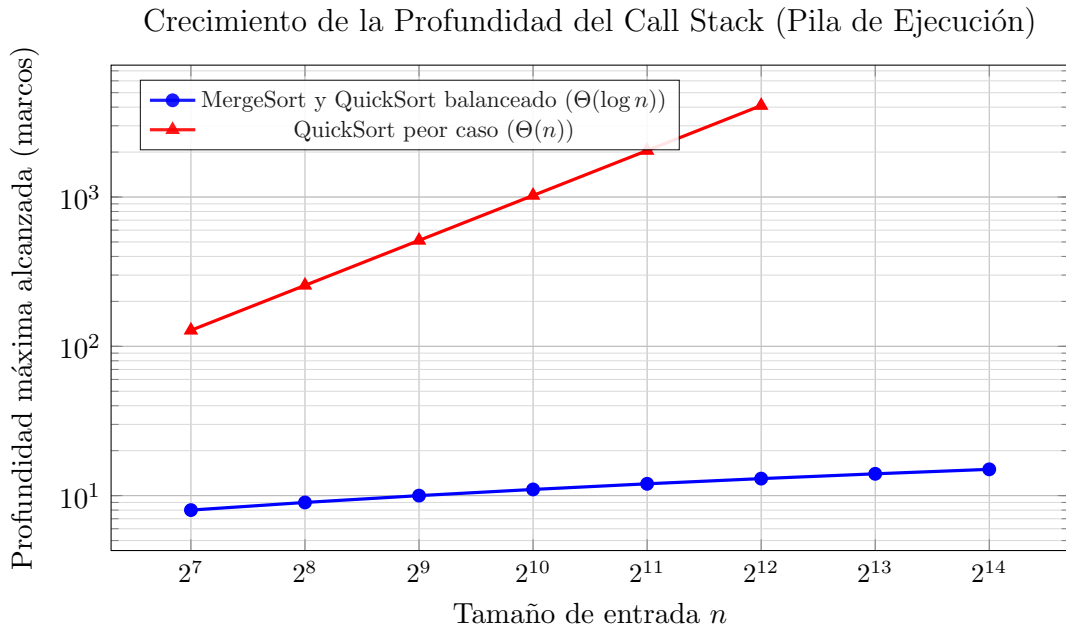


Figura 4: Evolución del consumo espacial en memoria de pila frente al crecimiento del tamaño del problema.

para MergeSort con $n = 16384$. Esto refleja la baja resolución de los métodos nativos y la recolección de basura asíncrona de la JVM. Sin embargo, teóricamente se demuestra que el arreglo auxiliar de MergeSort requiere exactamente $4n$ bytes de memoria dinámica para almacenar enteros, concluyendo el costo espacial $\mathcal{O}(n)$.

9. Discusión

Los resultados empíricos respaldan la utilidad de las ecuaciones de diferencias discretas como herramienta de predicción. El ajuste de mínimos cuadrados con un $R^2 \geq 0,99$ significa que el crecimiento experimental es perfectamente coherente con la tendencia asintótica

matemática.

QuickSort balanceado demostró ser sustancialmente más rápido que MergeSort en esta implementación, a pesar de compartir el orden asintótico $\Theta(n \log n)$. Esto se debe a que MergeSort realiza copias constantes hacia el arreglo auxiliar, lo que incrementa el tráfico de memoria y las operaciones de escritura. Sin embargo, el talón de Aquiles de QuickSort es su dependencia del pivote: un arreglo ordenado evaluado con pivote extremo degrada el rendimiento a cuadrático y colapsa la pila rápidamente. Esto demuestra la importancia de mitigar la recursión descontrolada mediante la aleatorización del pivote, la técnica de mediana de tres, o el uso de recursión de cola optimizada.

10. Limitaciones y posibles mejoras

10.1. Limitaciones

- La ejecución se realizó sobre una sola versión de la JVM y un único hardware físico.
- `System.nanoTime()` incluye ruido de contexto introducido por el sistema operativo.
- El método `System.gc()` no garantiza una recolección inmediata de la memoria dinámica.
- El análisis estadístico no incluye intercepto ni análisis detallado de residuos.
- El punto de desbordamiento de pila depende estrictamente del parámetro `-Xss` configurado.

10.2. Posibles mejoras

- Migrar las pruebas a la herramienta estándar **JMH (Java Microbenchmark Harness)** para controlar de manera rigurosa la eliminación de código muerto y optimizaciones JIT [7].
- Incorporar variaciones de pivote como mediana de tres e *IntroSort*.
- Medir de forma directa la asignación de memoria utilizando perfiladores nativos como *Java Flight Recorder* o *async-profiler*.

11. Conclusiones

La integración de recurrencias matemáticas e instrumentación experimental permitió caracterizar con precisión el comportamiento real de los algoritmos de ordenamiento. Los modelos de ecuaciones de diferencias se validaron con coeficientes de correlación $R^2 > 0,99$, demostrando la coherencia entre el análisis discreto y el software real. Se determinó que el consumo de la pila de llamadas es un factor de riesgo crítico en algoritmos con recursión lineal como QuickSort en peor caso, alcanzando puntos de colapso físico cerca de los 8000 elementos bajo condiciones controladas de memoria. La ingeniería de software de alto nivel debe, por tanto, integrar el análisis asintótico con la comprensión de la pila de ejecución y el hardware real.

Referencias

- [1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

- [2] Sedgewick, R., y Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.
- [3] Elaydi, S. (2005). *An Introduction to Difference Equations* (3rd ed.). Springer.
- [4] Kelley, W. G., y Peterson, A. C. (2001). *Difference Equations: An Introduction with Applications* (2nd ed.). Academic Press.
- [5] Graham, R. L., Knuth, D. E., y Patashnik, O. (1994). *Concrete Mathematics: A Foundation for Computer Science* (2nd ed.). Addison-Wesley.
- [6] Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley.
- [7] OpenJDK. *Java Microbenchmark Harness (JMH)*, documentación técnica del proyecto.

Anexo A. Instrucciones de ejecución

Requisitos: JDK 17 o superior; no se requieren dependencias externas.

Linux o macOS

```
chmod +x scripts/*.sh
./scripts/test.sh
./scripts/run.sh
```

Aclaración: las banderas deben ser `--repetitions 7 --warmup 3` para que los valores sean correctos de este experimento.

Windows

```
scripts\test.bat
scripts\run.bat
```

Prueba opcional de pila

```
java -Xss1m -cp out/main com.proyecto.recursivos.StackLimitProbe
    1000 500 20000
```

El benchmark genera `data/benchmark-results.csv`. Los resultados incluidos en este reporte son una corrida de referencia y pueden variar al ejecutarse en otro equipo.